Module 3 — Customization

Lesson 3.3

Hooks — the automation layer

Hooks let the harness run shell commands at lifecycle events. They are how you enforce policy, not how you ask Claude to remember things.

Why it matters

Hooks are the difference between "I told Claude to do X" and "X gets done." The harness runs hooks regardless of whether Claude remembers; they are deterministic. Anything that should *always* happen — formatting, security scans, blocking dangerous commits — belongs in a hook.

Common confusion: users ask Claude to "always run tests after edits." Memory and instructions can't guarantee that. A `PostToolUse` hook can.

The events

Event	Fires when	Typical use
`PreToolUse`	Before a tool runs	Block dangerous operations, enforce policy
`PostToolUse`	After a tool runs	Auto-format edited files, run typecheck
`UserPromptSubmit`	User sends a message	Log prompts, inject context, gate
`Stop`	Claude finishes a turn	Notify, log, save state
`SubagentStop`	A subagent finishes	Capture results, log
`Notification`	Claude wants attention	Send to Slack, beep, push notification
`SessionStart`	Session opens	Set up shell aliases, load env, print banner
`PreCompact`	Before context compaction	Snapshot state for later recall

(Exact event names may shift between versions. Confirm with the docs.)

Hook shape

In `settings.json`:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "prettier --write \"$CLAUDE_TOOL_INPUT_FILE_PATH\""
          }
        ]
      }
    ]
  }
}
```

Each entry has:

- `matcher` — which tool name(s) trigger it; regex
- `hooks` — list of commands to run; each gets the tool's input via env vars and stdin

The command runs in your shell. It receives:

- Environment variables prefixed `CLAUDE_TOOL_INPUT_*` (e.g., `CLAUDE_TOOL_INPUT_FILE_PATH`)
- The full tool input as JSON on stdin

Exit codes (the policy mechanism)

The hook's exit code determines what happens:

Exit code	Meaning
`0`	Continue (allow the tool call / proceed)
`2`	Block (cancel the tool call; Claude sees the error)
Other non-zero	Error (Claude is told the hook failed but it continues)

Code 2 is your enforcement lever. Use it for hard blocks.

Examples

Auto-format on edit

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "node -e 'const f=process.env.CLAUDE_TOOL_INPUT_FILE_PATH;
    if(/\\.(ts|tsx|js)$/.test(f)) require(\"child_process\").execSync(`prettier --write
    ${f}`)'"
        }]
      }
    ]
  }
}
```

Block commits that contain `TODO(no-commit)`

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{
          "type": "command",
          "command": "bash .claude/hooks/block-no-commit.sh"
        }]
      }
    ]
  }
}
```

Runnable version: examples/02-pre-commit-hook.

Notify Slack when Claude needs attention

```
{
  "hooks": {
    "Notification": [
      {
        "hooks": [{
          "type": "command",
          "command": "curl -s -X POST $SLACK_WEBHOOK -d '{\"text\":\"Claude needs you\"}'"
        }]
      }
    ]
  }
}
```

Snapshot context before compaction

```
{
  "hooks": {
    "PreCompact": [
      {
        "hooks": [{
          "type": "command",
          "command": "date '+%Y-%m-%d %H:%M:%S' >> .claude/compactions.log"
        }]
      }
    ]
  }
}
```

Hook scripts vs. inline commands

For anything beyond one line, put the hook in a script file:

```
.claude/
  hooks/
    block-no-commit.sh
    format-on-edit.sh
  settings.json
```

Then reference them by path. This keeps `settings.json` readable and lets you commit/version the hooks.

When to use a hook vs. an instruction

You want...	Use
A behavior that must always happen	Hook
A behavior to apply most of the time, with Claude judging when	CLAUDE.md instruction
A subagent to run on specific triggers	Hook (it can spawn the agent)
A reminder Claude can ignore if context warrants	CLAUDE.md

Hooks are deterministic; instructions are advisory. Pick the right tool.

Try it

1. Add a `PostToolUse` hook that runs `git status --short` after every `Edit`. Watch the output flow.
2. Add a `PreToolUse` `Bash` hook that blocks any command containing `rm -rf`. Try to make Claude run one and watch it get blocked.
3. Add a `SessionStart` hook that prints a one-line greeting with the project name.
4. Remove all three after the exercise — they were just for taste.

Gotchas

- **Hooks fire in your shell**, not Claude's sandbox. Path, env, and `which`-resolution are whatever your shell sees. Test outside Claude first.
- **A failing hook can silently block work.** If Claude starts complaining that tools "won't run," check hooks first.
- **`PreToolUse` blocking is the source of truth.** If a hook returns exit 2 on every `Bash`, Claude can't shell out at all.
- **Hooks run for *every* matching event** — they can multiply latency. Keep them fast (sub-second).
- **Hook configs reload at session start.** Edits to hooks don't apply mid-session.

Further reading

- examples/02-pre-commit-hook — runnable demo
- Official hooks docs: https://docs.claude.com/en/docs/claude-code/hooks
- 3.2 settings.json — where hooks live

Next

→ 3.4 Custom slash commands